

Notations for Time Complexity

→ O → big-oh ✓ → standard; → measures worst case complexity
 θ → theta → avg
 ω → omega → Best → $\rightarrow x$

for (int i = 1; i <= n; i++) {
 Sort (arr[i]);
}

Time Complexity $O(n \times 3) \rightarrow O(n)$
 constant $\rightarrow n$
 $\rightarrow x$
 $\rightarrow y$
 $\rightarrow z$

① avoid the constants
 ② avoid the lower values.

$h=3$
 $O(3 \times 3)$
 $= O(9)$

for (int i = 0; i < n; i++) {
 for (int j = 0; j < n; j++) {
 Sort (arr[i]);
 }
}

Total n times
 $i=0 (j=0 \rightarrow n-1)$
 $i=1 (j=0 \rightarrow n-1)$
 $i=2 (j=0 \rightarrow n-1)$
 $\vdots$
 $i=n-1 (j=0 \rightarrow n-1)$

$n \times n$
 $O(n^2)$

for (int i = 0; i < n; i++) {
 for (j = 0; j < n; j++) {
 for (k = 0; k < n; k++) {
 // given
 $O(n^3)$
 $\sqrt{n}$
 $\sqrt{n}$ times
 $O(\sqrt{n})$
 $O(\sqrt{n} \times n)$
 $O(n^{3/2})$
 $i=\sqrt{n}$ - n times

Bubble

$O(n^2)$

for (int i = 0; i < n-1; i++) {
 for (int j = 0; j < n-1-i; j++) {
 if (arr[j] > arr[j+1]) {
 // swap (arr[j], arr[j+1]);
 }
 }
}

$i=0 (j=0 \rightarrow n-1)$
 $i=1 (j=0 \rightarrow n-2)$
 $i=2 (j=0 \rightarrow n-3)$
 $\vdots$
 $i=n-1 (j=0 \rightarrow 0)$

$O(n \times n) \rightarrow O(n^2)$
 $n \rightarrow 10^6$
 $10^6 - 1 \times \sim 10^6$
 $10^6 - 2 \times \sim 10^6$

• While writing time complexity
 always take the worst case
 scenario.

Insertion Sort
 n times
 for (int i = 1; i < n; i++) {
 int val = arr[i];
 int j = i-1;
 while (j > 0 && arr[j] > val) {
 arr[j+1] = arr[j];
 j--;
 }
 arr[j+1] = val;
 }

val = 5
 val = 4
 [3, 5, 6, 2, 1]
 [1, 2, 3, 4, 5, 6]

```

for (int i=1; i<=n; i++) {
    int val = arr[i];
    int j = i-1;
    while (j >= 0 & arr[j] > val) {
        arr[j+1] = arr[j];
        j--;
    }
    arr[j+1] = val;
}

```

$j = -1$

$arr[-1+1] = 1$
 $arr[0] = 1$

$\hookrightarrow O(n^2)$

$\hookrightarrow O(n)$

if my array is already sorted.

1st element sorted

$[3, 5, 6, 2, 1]$

sorted

$[1, 2, 3, 5, 6]$

$j = n-1$

$i=4 \rightarrow val=1$
 $j=3$ while $arr[j] > arr[i]$
 $[2, 3, 5, 6]$
 $1 \rightarrow 6$
 $val=1$
 $arr[j] > val$

$[2, 3, 5, 6]$
 $val=1$
 $[1, 2, 3, 5, 6]$
 $j = -1$

i
 j
 $val = arr[i];$
 $arr[j] = arr[i];$
 $arr[j+1] = val$

$[3, 5, 6, 2, 1]$
 $2 < 6$
 $5 > 2$
 $j+1$
 $arr[j+1] = arr[j]$
 $j > 2$

$[3, 3, 5, 6, 1]$
 $j = -1$
 $j+1 = 0$
 $arr[0] = val$

$\hookrightarrow for (int i=0; i<\sqrt{n}; i++)$ increment

$\hookrightarrow while (n, i >= 0)$ divide
 $n = n/2$
 $n = 5 \rightarrow$
 $n = 36 \rightarrow 100$
 $O(\sqrt{n} + \log n)$
 $O(\sqrt{n})$

$O(n^2 + n)$
 $for (int i=0; i<n; i++)$
 $for (j=0; j<n; j++)$
 $O(n^2)$
 $for (int i=0; i<n; i++)$
 $O(n)$

overall T.C
 $\rightarrow O(n^2) + O(n)$

$\hookrightarrow O(n^2 + n)$
 $\hookrightarrow O(n^2)$

$n = 10^4 \rightarrow 10000$
 $n^2 = 10^8 \approx (10^4)^2$

$$O(3n^3 + 2n^2 + n + 2) \rightarrow O(n^3)$$

Algo (code)

→ Always mention the worst case Time Complexity.

Insertion Sort

Best Case
→ $O(n)$

Average Case
 $O(n^2)$

Worst Case ✓
 $O(n^2)$ → [5, 4, 3, 2, 1]

Bubble Sort $O(n^2)$

$O(n^2)$

$O(n^2)$ while
for
 $n + (n-1) + (n-2) + \dots$

Selection Sort $O(n^2)$

$O(n^2)$

$O(n^2) \approx n^2$

[3, 4, 1, 2]
X
while